

Git-Cheatsheet für Hugo-Projekte (Windows + GitHub)

Erstellt für: Swetoslaw Beltschew

■ Zielsetzung

Dieses Cheatsheet unterstützt dich bei der täglichen Arbeit mit deinem Hugo-Blogprojekt in Verbindung mit GitHub.

■■ Projekt-Setup (einmalig)

```
Set-Location "C:\Pfad\zu\HugoProjekt"
git init
git add .
git commit -m "Initialer Commit - bestehendes Hugo-Projekt"
git branch -M main
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

■ Täglicher Workflow

```
Set-Location "C:\Pfad\zu\HugoProjekt"
git status
git add .
git commit -m "Beitrag vom 2025-10-22 ergänzt"
git push
```

■ Änderungen vom GitHub-Repo holen

```
git pull
```

■ Branch erstellen (z. B. für Testbeitrag)

```
git checkout -b testbeitrag
# Änderungen machen...
git checkout main
git merge testbeitrag
```

■ Theme-Probleme lösen (Submodul entfernen)

```
git rm --cached -r themes\hugo-serif-theme
Remove-Item -Recurse -Force themes\hugo-serif-theme\.git
git add themes\hugo-serif-theme
git commit -m "Theme vollständig integriert - kein Submodul mehr"
```

■ Empfohlene .gitignore (Auszug)

```
public/
resources/
.vscode/
node_modules/
*.log
*.tmp
*.swp
.DS_Store
```

■ Nützliche Git-Konfigurationen

```
git config core.preloadindex true
git config core.fscache true
```

■ Nützliche Git-Befehle

```
git status          # zeigt Änderungen
git log             # Commit-Historie
git diff            # Unterschiede anzeigen
git checkout <name> # Branch/Commit wechseln
git restore <file>  # Datei zurücksetzen
git rm --cached <pfad> # Datei aus Git entfernen
```

■ Empfehlung

- Nach jedem Arbeitstag oder größerem Schritt:
git add . → git commit → git push
- Fehler vermeiden:
 - Niemals public/ oder resources/ mitversionieren
 - .gitignore aktuell halten
 - Submodule vermeiden, wenn Themes lokal integriert sind

■ Dein Remote-Repository

<https://github.com/sweti61/firma-mediator-sweti>